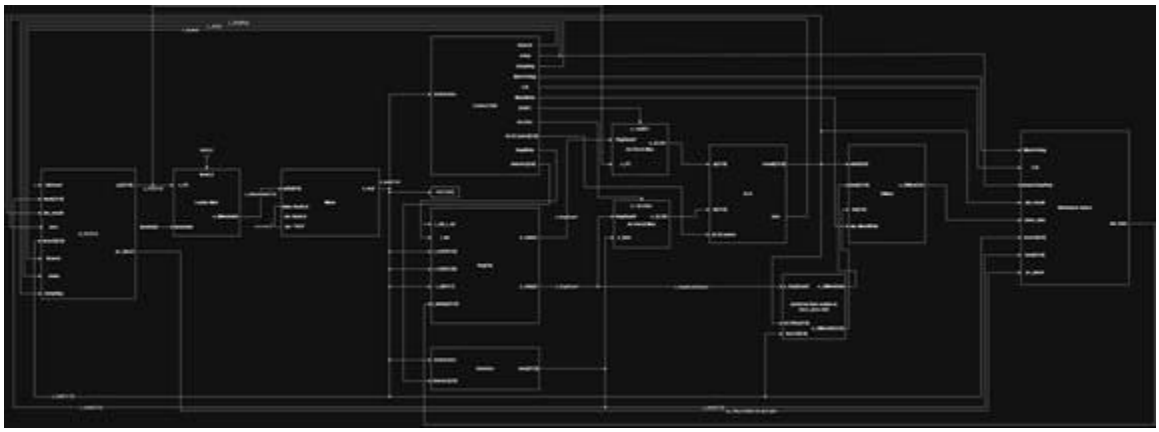
CprE 3810: Computer Organization and Assembly-Level Programming

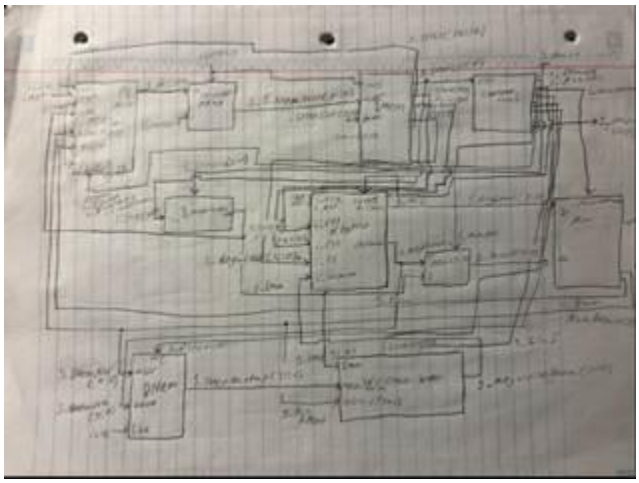
Project Part 1 Report

Team Members: Graham Hynnek, Alexander Anderson, Andres Garcia

Project Teams Group #:__ E_08 ___
Refer to the highlighted language in the project 1 instruction for the context of the following questions.
[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.
Digital:



HandDrawn:



Our processor is a single cycle datapath that includes extra hardware required per full the instruction set. Our fetch unit contains the PC register and a PC + 4 adder as well as a PC + immediate adder. The 3-way mux selects the next PC. The instruction goes to the control unit, the reigster file, and the immediate generator. Our ALU has an A input and

that comes from a 2:1 mux, rs1, or the PC for the auipc. The B input comes from a second 2:1 mux, rs2, or the immediate "ALUSrc". Our ALU will result in the data memory address. When it stores, store merge logic builds the words and then writes for sb and sh thus then setting the byte enables. Our writeback select block chooses which is written to the rd, the ALU results, our load data, or zero extended for things such as our lb, lh, lbu, lhu, our immediate for the lui, or PC + 4 for our jal, and jalr. There is also a wifi decoder that is driving our toolflow's halt signal.

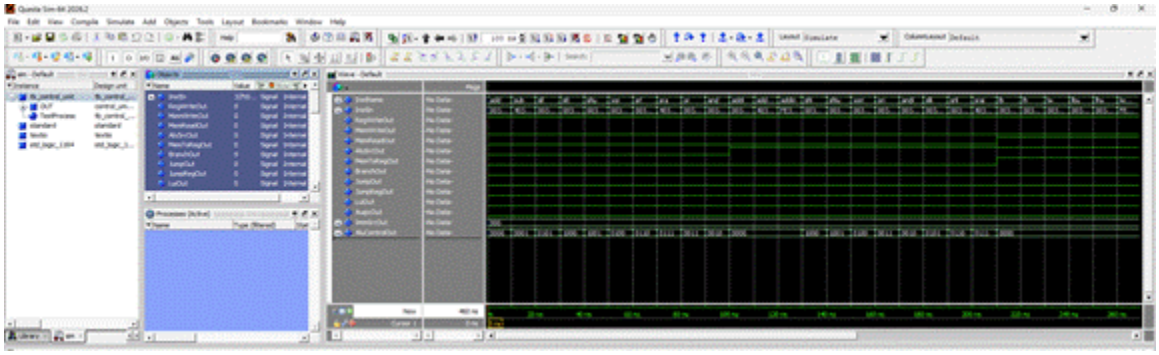
[Part 3.1.a.] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the *N* control signals needed by your datapath implementation. The end result should be an *N***M* table where each row corresponds to the output of the control logic module for a given instruction.

Instruction	Format	Opl[6]	Opl[5]	Opl[4]	Opl[3]	Opl[2]	Opl[1]	Opl[0]	F3[2]	F3[1]	F3[0]	F7[6]	F7[5]	F7[4]	F7[3]	F7[2]	F7[1]	F7[0]	RegWrite	MemWrite	MemRead	ALUSrc	AUIPC (ALUSrcA)	MemToReg	Branch	Jump	JumpReg	LUI	ImmSrc[2]	ImmSrc[1]	ImmSrc[0]	ALUCtrl[3]	ALUCtrl[2]	ALUCtrl[1]	ALUCtrl[0]	Instruction & Control Signal Purpose / RTL Description
add	R-Type	0	1	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	rd = rs1 + rs2 (R-type addition)
sub	R-Type	0	1	1	0	0	1	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	rd = rs1- rs2 (R-type subtraction)
and	R-Type	0	1	1	0	0	1	1	1	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	rd = rs1 & rs2 (Bitwise AND)
or	R-Type	0	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	rd = rs1 | rs2 (Bitwise OR)
xor	R-Type	0	1	1	0	0	1	1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	rd = rs1 ^ rs2 (Bitwise XOR)
sll	R-Type	0	1	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	rd = rs1 << rs2[4:0] (Logical left shift)
srl	R-Type	0	1	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	rd = rs1 >> rs2[4:0] (Logical right shift)
sra	R-Type	0	1	1	0	0	1	1	0	0	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	rd = rs1 >>> rs2[4:0] (Arithmetic right shift)
slt	R-Type	0	1	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	rd = (rs1 < rs2) ? 1 : 0 (Signed set less than)
sltu	R-Type	0	1	1	0	0	1	1	0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	rd = (rs1 < urs2) ? 1 : 0 (Unsigned set less than)
addi	I-Type	0	0	1	0	0	1	1	0	0	0	-	-	-	-	-	-	-	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	rd = rs1 + imm_i (Add immediate)
andi	I-Type	0	0	1	0	0	1	1	1	1	1	-	-	-	-	-	-	-	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	rd = rs1 & imm_i (AND immediate)
ori	I-Type	0	0	1	0	0	1	1	1	1	0	-	-	-	-	-	-	-	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	rd = rs1 | imm_i (OR immediate)
xori	I-Type	0	0	1	0	0	1	1	1	0	0	-	-	-	-	-	-	-	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	rd = rs1 ^ imm_i (XOR immediate)
slli	I-Type	0	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	rd = rs1 << shamt (Shift left logical immediate)
srli	I-Type	0	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	rd = rs1 >> shamt (Shift right logical immediate)
srai	I-Type	0	0	1	0	0	1	1	0	0	1	0	1	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	rd = rs1 >>> shamt (Shift right arithmetic immediate)
slti	I-Type	0	0	1	0	0	1	1	0	1	0	-	-	-	-	-	-	-	1	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	rd = (rs1 < imm_i) ? 1 : 0 (Set less than immediate)
sltiu	I-Type	0	0	1	0	0	1	1	0	1	1	-	-	-	-	-	-	-	1	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	rd = (rs1 < u imm_i) ? 1 : 0 (Set less than imm unsigned)
lb	I-Load	0	0	0	0	0	1	1	0	0	0	-	-	-	-	-	-	-	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	rd = SignExt(Mem[rs1 + imm_i][7:0]) (Load byte)
lh	I-Load	0	0	0	0	0	1	1	0	0	1	-	-	-	-	-	-	-	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	rd = SignExt(Mem[rs1 + imm_i][15:0]) (Load halfword)
lw	I-Load	0	0	0	0	0	1	1	0	1	0	-	-	-	-	-	-	-	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	rd = Mem[rs1 + imm_i][31:0] (Load word)
lbu	I-Load	0	0	0	0	0	1	1	1	0	0	-	-	-	-	-	-	-	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	rd = ZeroExt(Mem[rs1 + imm_i][7:0]) (Load byte unsigned)
lhu	I-Load	0	0	0	0	0	1	1	1	0	1	-	-	-	-	-	-	-	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	rd = ZeroExt(Mem[rs1 + imm_i][15:0]) (Load halfword unsigned)
sb	S-Type	0	1	0	0	0	1	1	0	0	0	-	-	-	-	-	-	-	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	Mem[rs1 + imm_s][7:0] = rs2[7:0] (Store byte)
sh	S-Type	0	1	0	0	0	1	1	0	0	1	-	-	-	-	-	-	-	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	Mem[rs1 + imm_s][15:0] = rs2[15:0] (Store halfword)
sw	S-Type	0	1	0	0	0	1	1	0	1	0	-	-	-	-	-	-	-	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	Mem[rs1 + imm_s][31:0] = rs2[31:0] (Store word)
beq	B-Type	1	1	0	0	0	1	1	0	0	0	-	-	-	-	-	-	-	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	if (rs1 == rs2) PC += imm_b (Branch equal)
bne	B-Type	1	1	0	0	0	1	1	0	0	1	-	-	-	-	-	-	-	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	if (rs1 != rs2) PC += imm_b (Branch not equal)
blt	B-Type	1	1	0	0	0	1	1	1	0	0	-	-	-	-	-	-	-	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	if (rs1 < rs2) PC += imm_b (Branch less than signed)
bge	B-Type	1	1	0	0	0	1	1	1	0	1	-	-	-	-	-	-	-	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	if (rs1 >= rs2) PC += imm_b (Branch greater/equal signed)
bltu	B-Type	1	1	0	0	0	1	1	1	1	0	-	-	-	-	-	-	-	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	if (rs1 < u rs2) PC += imm_b (Branch less than unsigned)
bgeu	B-Type	1	1	0	0	0	1	1	1	1	1	-	-	-	-	-	-	-	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	if (rs1 >= u rs2) PC += imm_b (Branch greater/equal unsigned)
lui	U-Type	0	1	1	0	1	1	1	-	-	-	-	-	-	-	-	-	-	1	0	0	1	0	0	0	0	0	1	0	1	1	0	0	0	0	rd = imm_u (Load upper immediate: inst[31:12] << 12)
auipc	U-Type	0	0	1	0	1	1	1	-	-	-	-	-	-	-	-	-	-	1	0	0	1	1	0	0	0	0	1	0	1	1	0	0	0	0	rd = PC + imm_u (Add upper immediate to PC)
jal	J-Type	1	1	0	1	1	1	1	-	-	-	-	-	-	-	-	-	-	1	0	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	rd = PC + 4; PC = PC + imm_j (Jump and link)
jalr	I-Jump	1	1	0	0	1	1	1	0	0	0	-	-	-	-	-	-	-	1	0	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	rd = PC + 4; PC = (rs1 + imm_i) & ~1 (Jump and link reg)
wfi	System	1	1	1	1	0	0	1	1	1	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Halt simulation (s_Halt = 1 when funct12 = 000100000101)

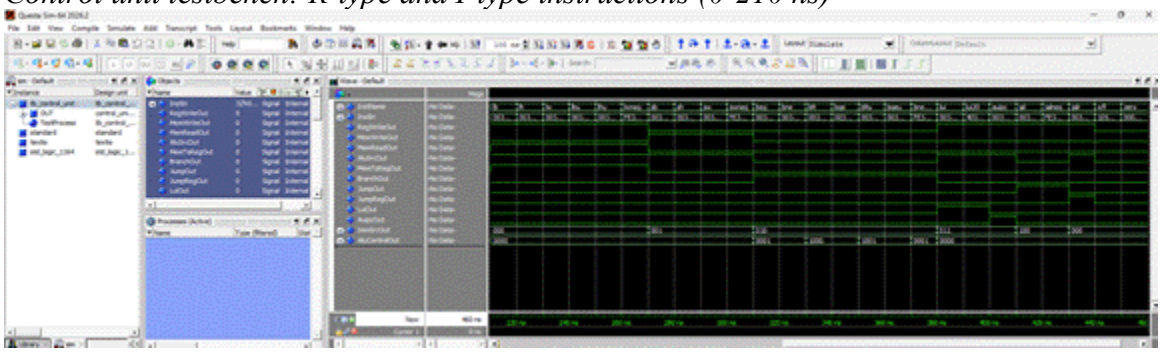
Our ControlSignals.xksx contains one row per each instruction which is 37 instructions total. It contains opcode, funct3, and funct7 split into single bit coloms. The Jump, JumpReg, LUI, AUIPC, ImmSrc, and the ALUControl. We have also added additional signals that were not initially included within the textbook design, and these are Jump and JumpRreg and these select the jal or the jalr target. LUI to which is used to send a immiediate straight to the writeback. AUIPC is used to switch the ALU input A to the PC. Last but not least, ImmSrc in which needs 3 bits to cover five immediate formats.

[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).

Our control unit is a combination of dataflow VHDL. It is used to decode the opcode into one flag per instruction class. Each output is OR of these flags mentioned. Our ALUcontrol uses funct3 as well as funct7 with bit 5 of funct7 used to separate the add and the sub as well as the srl and sra. The branches use sub such as beq, bne, slt, blt, bge, sltu, bltu, and bgeu. These then load and store jalr, and auipc and uses add.



Control unit testbench: R-type and I-type instructions (0-210 ns)



Control unit testbench: loads, stores, branches, lui/auipc, jal/jalr, wfi (210-460 ns)

```
VSIM 57> add wave /tb_control_unit/RegWriteOut /tb_control_unit/MemWriteOut /tb_control_unit/MemReadOut /tb_control_unit/AluSrcOut /tb_control_unit/MemToRegOut
VSIM 58> add wave /tb_control_unit/BranchOut /tb_control_unit/JumpOut /tb_control_unit/JumpRegOut /tb_control_unit/LuiOut /tb_control_unit/AuipcOut
VSIM 59> add wave -radix binary /tb_control_unit/ImmSrcOut /tb_control_unit/AluControlOut
VSIM 60> run -all
# ** Note: tb_control_unit: ALL 46 TESTS PASSED
#    Time: 460 ns  Iteration: 0  Instance: /tb_control_unit
VSIM 61> wave zoom range 0ns 210ns
# (0 ns) (210 ns)

VSIM 62>
```

tb_control_unit transcript: all 46 tests passed

[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

Our PC is 32 flip flops in which bit 22 sets on reset and the rest clear. This gives you 0x00400000. Two ripple carry adders are used to compute PC + 4 and PC + Imm as does a mux also chooses the next PC.

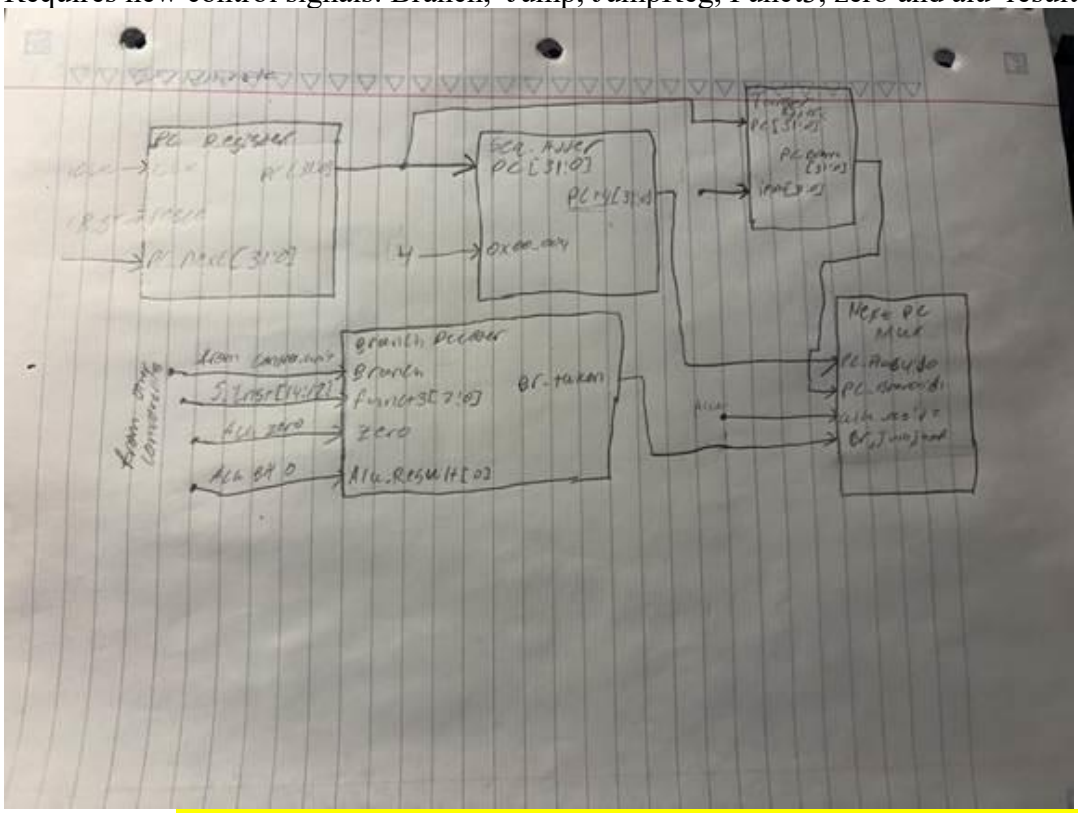
00: PC + 4
01: PC + Imm
10: ALU result with bit 0 = 0

Our branch condition will come from the ALU and result in 0 as for the others such as for the others slt, sltu. Funct3(2) picks between the two, funct3(0) inverts the condition, which turns beq into bne, blt into bge, and bltu into bgeu.

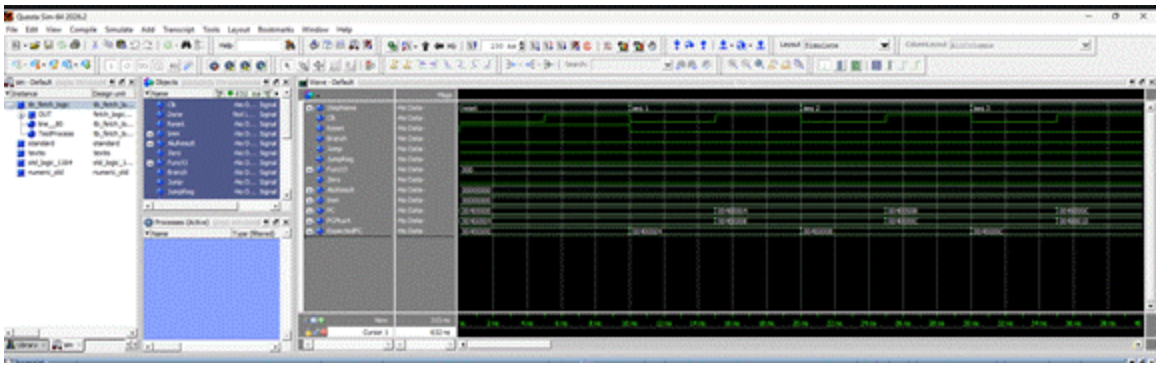
The new control signals now are Branch, Jump, and JumpReg. Funct3 is routed to teh fetch. Then the ALU's zero and result (0) move to fetch, the writeback mux gets a PC + 4 input.

[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?

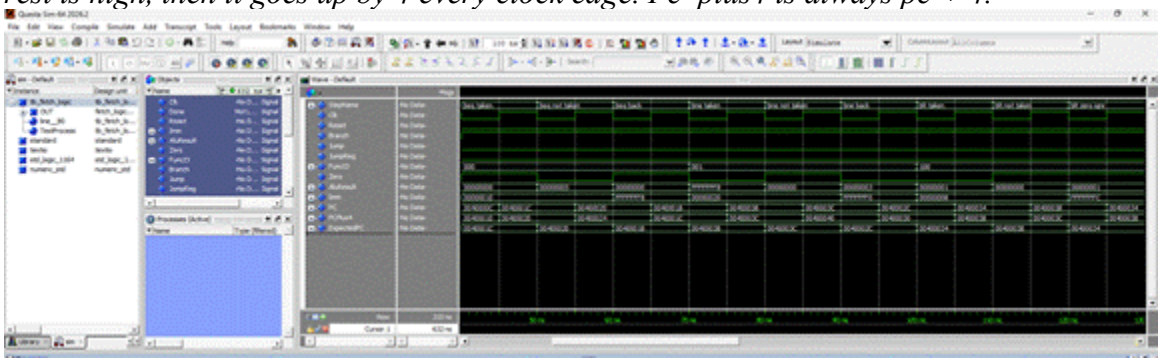
Requires new control signals: Branch, Jump, JumpReg, Funct3, zero and alu result



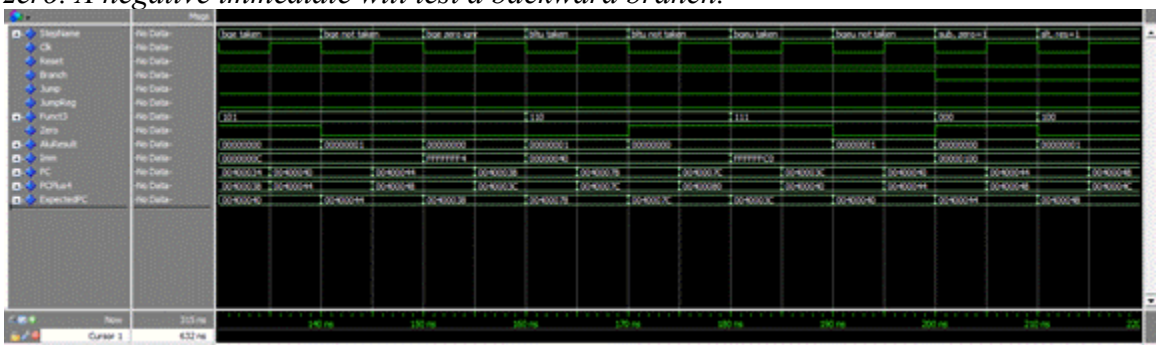
[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.



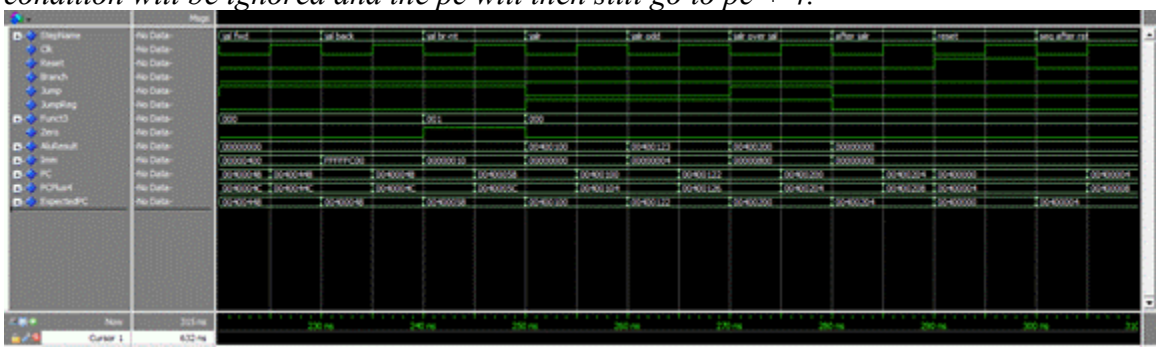
Fetch logic testbench: reset to 0x00400000, then PC + 4, pc holds 0x00400000 while the rest is high, then it goes up by 4 every clock edge. Pc plus4 is always pc + 4.



Fetch logic testbench: beq, bne, blt, A taken branch will make the next pc = pc + imm and a branch that isnt taken will give pc + 4 . The blt uses the alu_result(0) instead of zero. A negative immediate will test a backward branch.



Fetch logic testbench: bge, bltu, bgeu, are taken when the the alt and altu bit is 0, which shows us that funct3(0) inversion is working correctly. When Branch = 0 then a true condition will be ignored and the pc will then still go to pc + 4.



Fetch logic testbench: jal, jalr, and reset

```
SIM 84> add wave -radix hex /tb_fetch_logic/AluResult /tb_fetch_logic/Imm
SIM 85> add wave -radix hex /tb_fetch_logic/PC /tb_fetch_logic/PCPlus4 /tb_fetch_logic/ExpectedPC
SIM 86> run -all
  ** Note: tb_fetch_logic: ALL 62 TESTS PASSED
     Time: 310 ns  Iteration: 0  Instance: /tb_fetch_logic

SIM 87>
```

ow: 315 ns Delta: 1 | sim:/tb_fetch_logic

tb_fetch_logic transcript: all 62 tests passed

[Part 3.3.1.(a)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does RISC-V not have a `sla` instruction?

Srl Sra the difference between the two is that srl fills the bit positions with zero while sra shifts it but fills the upper bit positions with the sign bit, Risc-V doesn't have a sla because it is identical to sll at the bit level

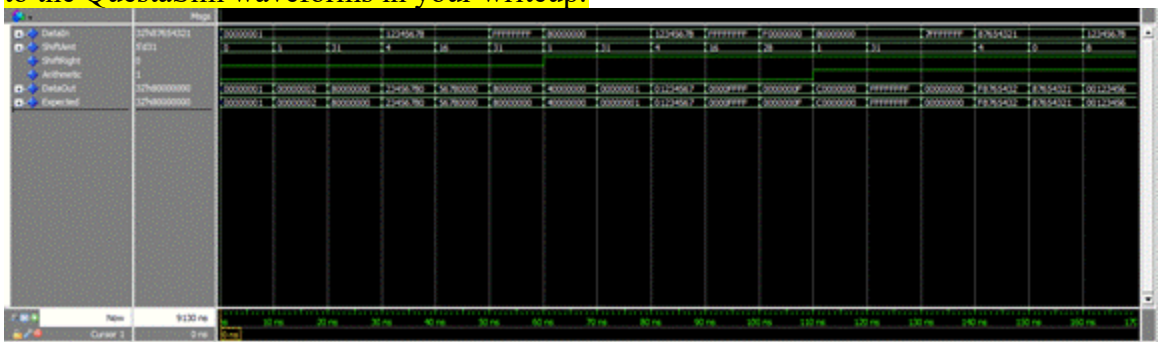
[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.

The barrel shifter implements the right shift using 2 to 1 muxes controlled by shamt[4:0] to support both types of shifts right shifts compute a single fillbit when arithmetic shift control signal is 0 fill_bit <= 0 when it is 1 the fill bit is data in (31)

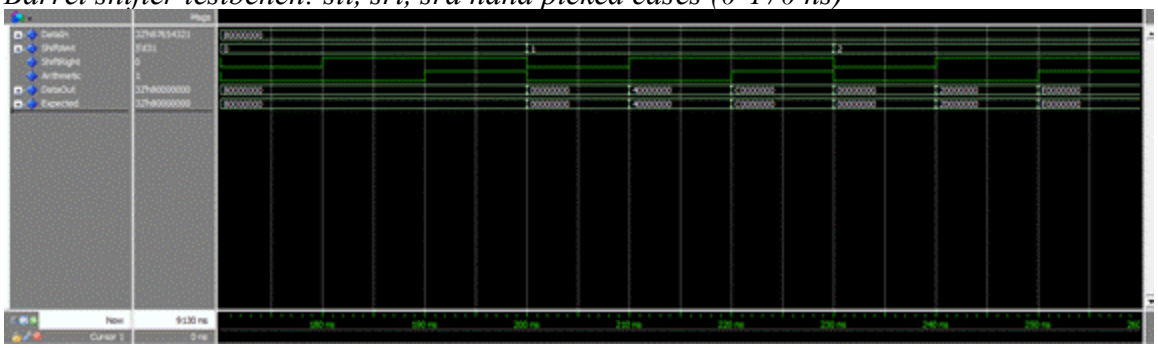
[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations.

A left shift would be the same as flipping the order of the 32 bit input order, when the left_shift = 1 the input operand bits are flipped to [0:31]

[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.



Barrel shifter testbench: sll, srl, sra hand picked cases (0-170 ns)



Barrel shifter testbench: 0x80000000 shifted by 0, 1, 2 (sll, srl, sra)

```
VSIM 98> run -all
# ** Note: tb_barrel_shifter: ALL 913 TESTS PASSED
#    Time: 9130 ns  Iteration: 0  Instance: /tb_barrel_shifter
VSIM 99> wave zoom range 0ns 170ns
# {0 ns} {170 ns}

VSIM 100>
```

tb_barrel_shifter transcript: all 913 tests passed

[Part 3.3.2.(a)] In your writeup, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

We made the imm gen and the write back unit we used the given files to hook up the required signals, One design decision we made was to use the alu unit to calulate the memory address location with s_*PC* + *s*_Imm with the U_ALUSRCA_MUX

[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

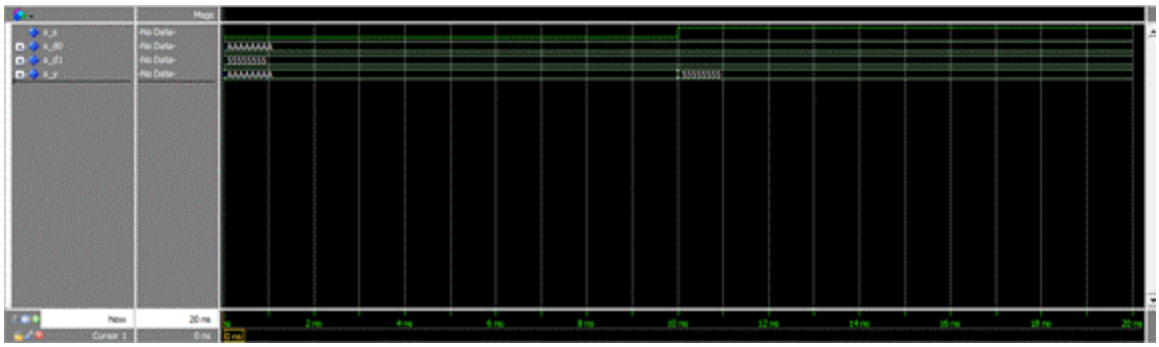


Immediate generator testbench: one real instruction of each format (I, S, B, U, J)

```
# Loading ieee.numeric_std(body)
# Loading work.tb_imm_gen(behavior)#1
# Loading work.imm_gen(dataflow)#1
VSIM 117> config wave -signalnamewidth 1
VSIM 118> add wave -radix hex /tb_imm_gen/InstIn
VSIM 119> add wave -radix binary /tb_imm_gen/ImmSrc
VSIM 120> add wave -radix hex /tb_imm_gen/ImmOut /tb_imm_gen/Expected
VSIM 121> run -all
# ** Note: tb_imm_gen: ALL 105 TESTS PASSED
#    Time: 1050 ns  Iteration: 0  Instance: /tb_imm_gen
VSIM 122> wave zoom range 0ns 90ns
# {0 ns} {90 ns}

VSIM 123>
Now: 1,050 ns  Delta: 1        sim:/tb_imm_gen
```

tb_imm_gen transcript: all 105 tests passed



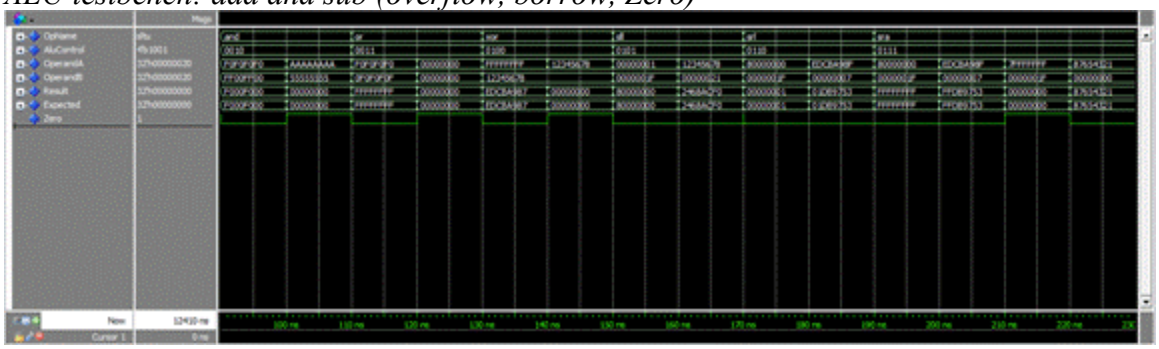
2:1 mux testbench: y = d0 when s = 0, y = d1 when s = 1

[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?

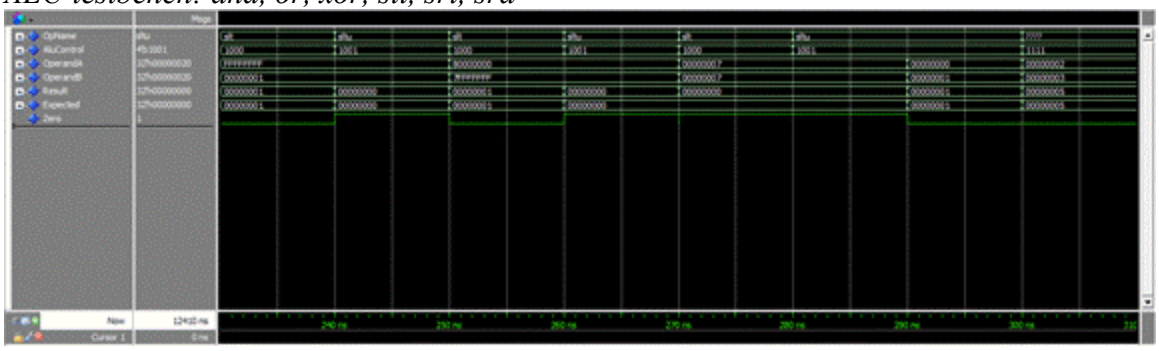
[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.



ALU testbench: add and sub (overflow, borrow, Zero)



ALU testbench: and, or, xor, sll, srl, sra



ALU testbench: slt vs sltu, and unused ALUControl code

```
VSIM 107> add wave -radix binary /tb_alu/AluControl
VSIM 108> add wave -radix hex /tb_alu/OperandA /tb_alu/OperandB /tb_alu/Result /tb_alu/Expected
VSIM 109> add wave /tb_alu/Zero
VSIM 110> run -all
# ** Note: tb_alu: ALL 1241 TESTS PASSED
#    Time: 12410 ns  Iteration: 0  Instance: /tb_alu
VSIM 111> wave zoom range 0ns 90ns
# (0 ns) (90 ns)

VSIM 112>
Now: 12,410 ns  Delta: 1          sim:/tb_alu
```

tb_alu transcript: all 1241 tests passed

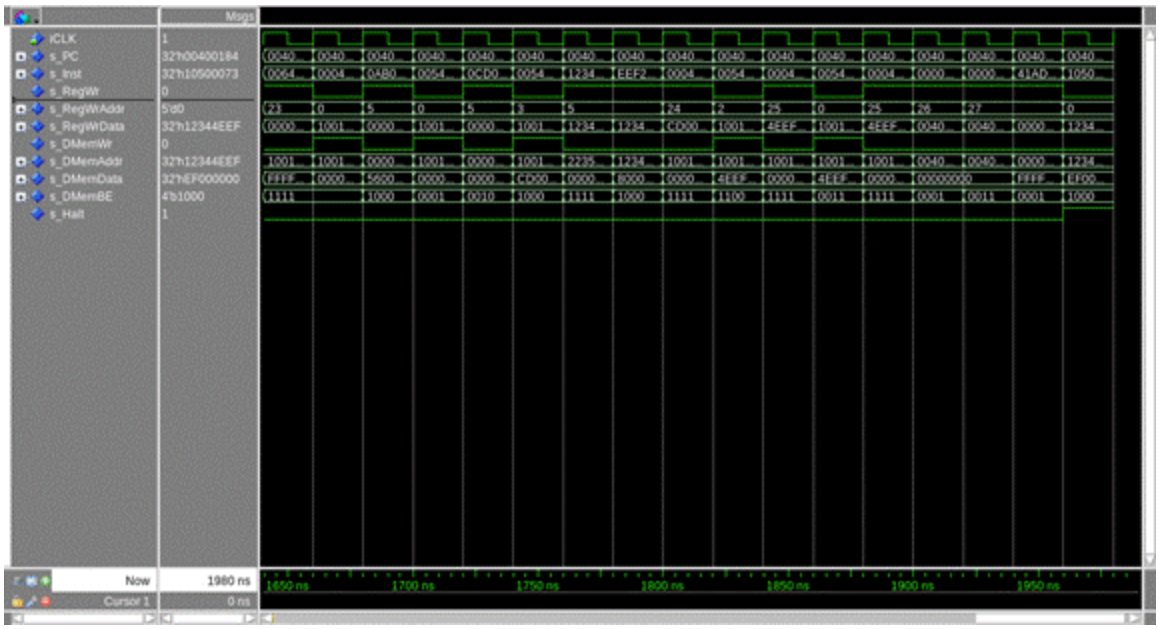
[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.

[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

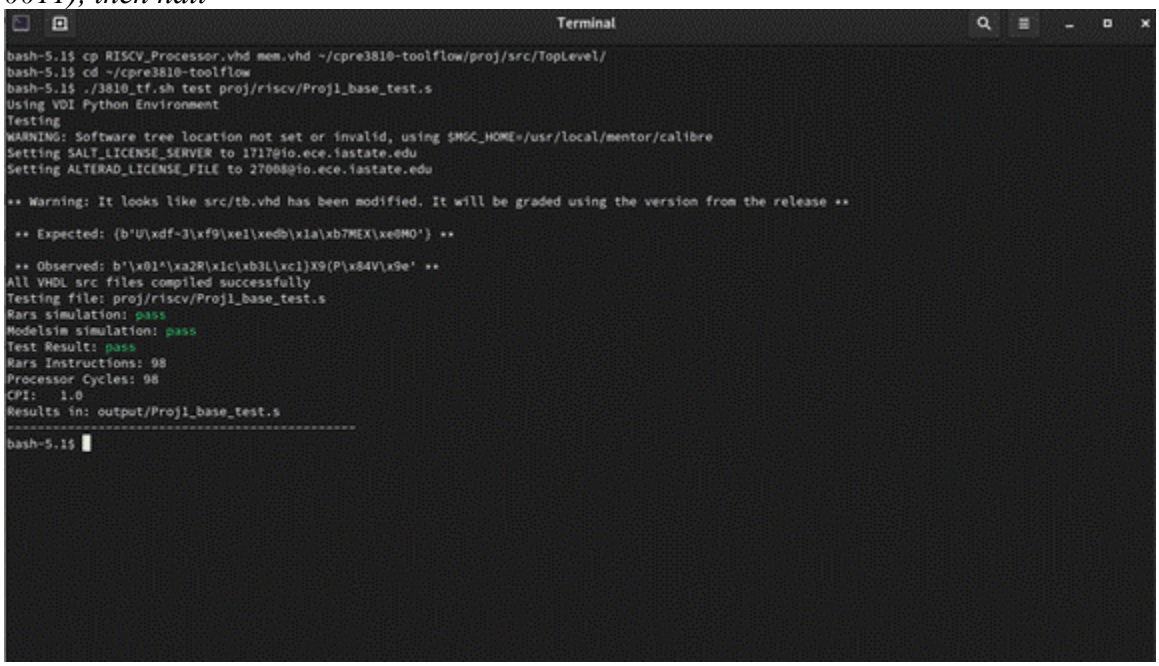
[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.



Proj1_base_test.s: first instructions, each result written to a register and used by the next instruction



Proj1_base_test.s: sw, sb and sh stores with byte enables (1111, 0001, 1000, 1100, 0011), then halt

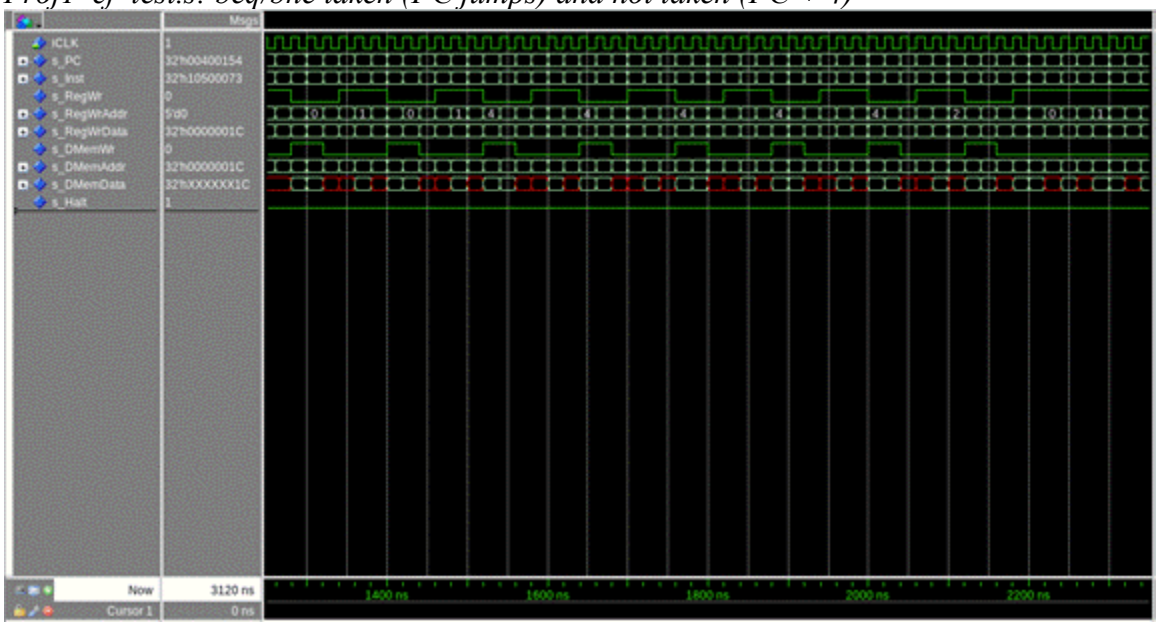


Proj1_base_test.s toolflow result: pass, 98 instructions, CPI 1.0

[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.



Proj1_cf_test.s: beq/bne taken (PC jumps) and not taken (PC + 4)



Proj1_cf_test.s: recursive Rsum calls, two stack stores per call

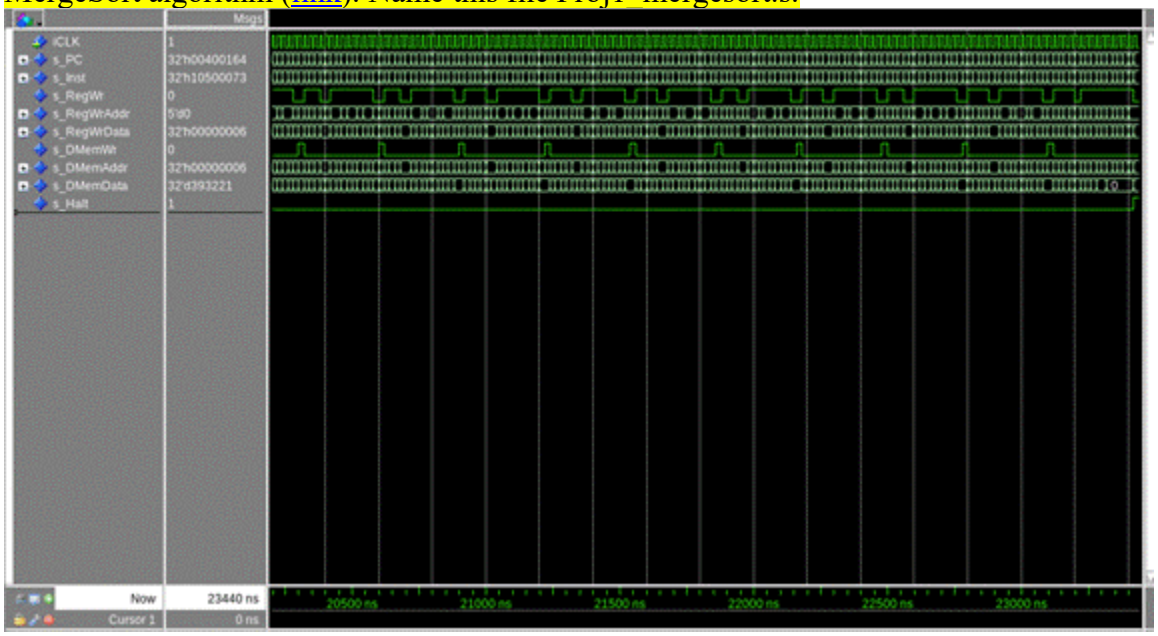


Proj1_cf_test.s: two Rsum calls, sp goes down by 8 and ra / n are pushed each call

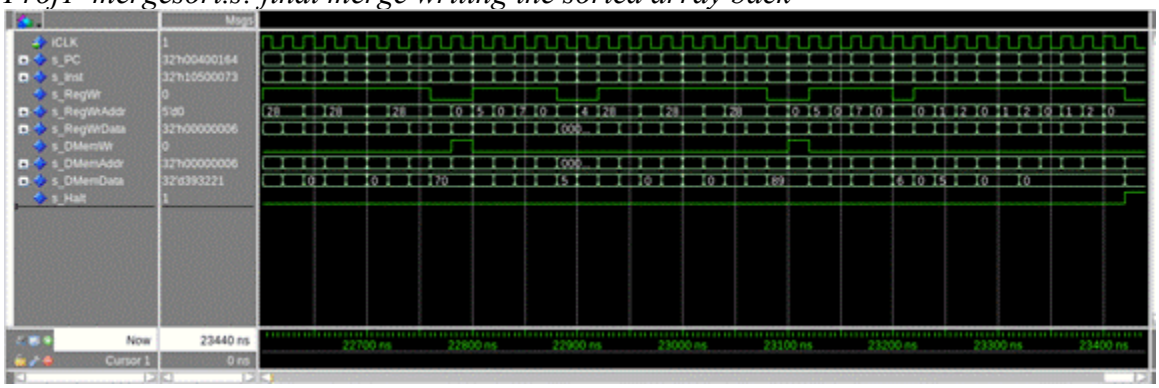
```
Testing file: proj/riscv/Proj1_cf_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
tsclient
Instructions: 155
Processor Cycles: 155
CPI:   1.0
Results in: output/Proj1_cf_test.s
```

Proj1_cf_test.s toolflow result: pass, 155 instructions, CPI 1.0

[Part 4.c] Create and test an application that sorts an array with *N* elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.



Proj1_mergesort.s: final merge writing the sorted array back

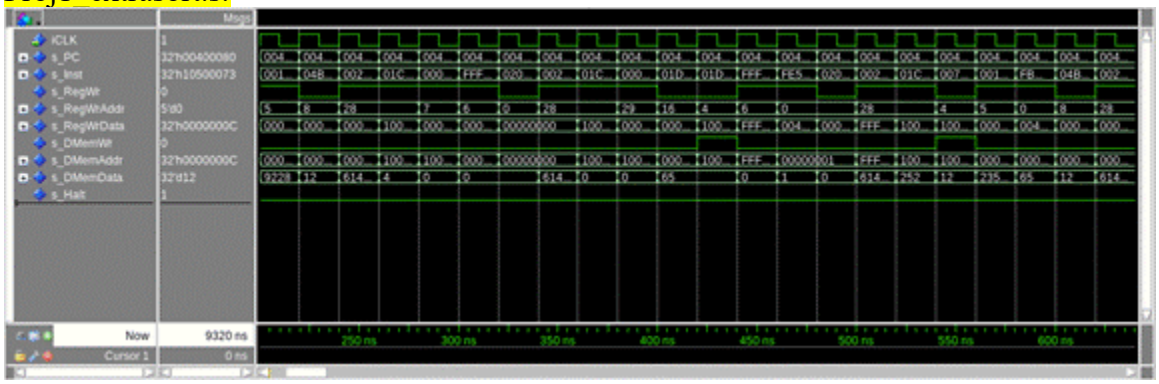


Proj1_mergesort.s: last stores of the final merge, then halt

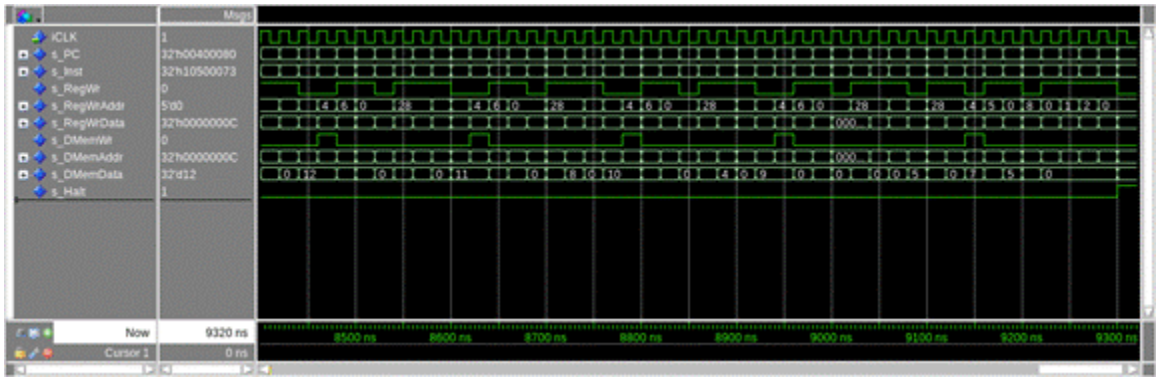
```
Testing file: proj/riscv/Proj1_mergesort.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 1171
Processor Cycles: 1171
CPI:    1.0
Results in: output/Proj1_mergesort.s
```

Proj1_mergesort.s toolflow result: pass, 1171 instructions, CPI 1.0

[Part 4.d] For teams of 3: Create and test an application that sorts an array with N elements using an algorithm that is NOT Bubble Sort or Merge Sort. Name this file Proj1_extrasort.s.



Proj1_extrasort.s (insertion sort): first insertion, 65 shifted right and 12 stored in front of it



Proj1_extrasort.s: last insertion shifting the larger values right, then halt

```
Testing file: proj/riscv/Proj1_extrasort.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 465
Processor Cycles: 465
CPI:   1.0
Results in: output/Proj1_extrasort.s
-----------------------------------------------
```

Proj1_extrasort.s toolflow result: pass, 465 instructions, CPI 1.0

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

Critical Path

